# Ders 6 Çalışma Özeti

## Ders 6 Çalışma Özeti

### Genel Konular

- Ders 5 Nisan 2023 Çarşamba günü yapıldı. Hoca dersi üniversitedeki sınıftan online verdi. Elektrik ve bağlantı kesintileri yüzünden ders iki kez bölündü.
- Nesneye yönelik çözümleme (object-oriented analysis) süreci tekrar edildi:
  - Uygulama alanının çözümlenmesi (domain analysis).
  - Gereksinim analizinin adımları: işi anlama (inception), bilgi toplama (elicitation), modelleme (elaboration), pazarlık (negotiation).
  - Kullanım öyküsü (use case) belgesi ve kullanım şeması (use case diagram).
  - `include` ve `extend` ilişkileri.
  - Aday sınıfların bulunması ve kavramsal sınıf şeması (conceptual class diagram).
  - SRS belgesi (Software Requirements Specification).
- Tasarım şemaları:
  - Ardışıl şema (sequence diagram) kısaca hatırlatıldı.
  - Etkinlik şeması (activity diagram): çatal (fork), birleşme (join), kulvarlar (swimlanes).
  - Durum şeması (state diagram): uçak yaşam döngüsü ve kasa satış örnekleri.
  - Tasarım sınıf şeması (design class diagram).
- Tasarım kalıpları (design patterns):
  - Yaratıcı (Creator)
  - Bilginin uzmanı (Information Expert)
  - Az bağımlılık (Low Coupling)
  - Denetçi (Controller)
  - Yüksek uyum (High Cohesion)
  - Çok şekillilik (Polymorphism)
  - Yapay sınıf (Pure Fabrication)
  - Dolaylılık (Indirection)
  - Değişimlerden korunma (Protected Variations)
- Yeniden kullanılabilirlik (reusability) kavramı ve tasarım kalıplarıyla ilişkisi.
- Etkileşim şemaları (interaction diagrams): ardışıl şema ile işbirliği şeması (collaboration diagram) arasındaki fark.
- Proje konuları ve gelecek haftaki uygulama dersi duyuruldu.

### Hocanın Özellikle Vurguladığı Kısımlar

- **Analiz ile tasarım farklıdır.** Analiz problemi anlamaktır: "ne yapılacak?" sorusunu cevaplar. Tasarım ise çözümün nasıl kurulacağını gösterir. Analizin çıktısı SRS belgesidir. Tasarım bu belgeye dayanarak başlar.
- **Kullanım öyküsü belgesine yalnız ilgi alanına giren bilgi yazılır.** Bizden istenmeyen konular belgeye konmaz.
- **Kavramsal sınıf şemasında işlemler (metotlar) yazılmaz.** Yalnız sınıf adı, nitelikler ve ilişkiler gösterilir. Metotlar tasarım sınıf şemasında eklenir.
- **Etkinlik şeması**, işlemlerin farklı ortamlarda ve farklı aktörlerde nasıl yürüdüğünü gösterir. Fork ve join yapısı bu yüzden önemlidir.
- **Durum şeması**, bir nesnenin ya da aktörün yaşamı boyunca hangi durumlardan geçtiğini gösterir.
- **Tasarım kalıplarını ardışıl şemalarda gösterin.** Sınavda verilen bir tasarıma uygun kalıbı (denetçi, yaratıcı, uzman vb.) yerleştirmeniz istenebilir.

- **Yapay sınıf kalıbını az kullanın.** Çok fazla yapay sınıf, tasarımı gerçek dünyadan uzaklaştırır ve anlaşılırlığı azaltır.
- **İyi tasarım = yüksek uyum + az bağımlılık.** Bu kural yapısal tasarımda da, nesneye yönelik tasarımda da geçerlidir.
- Nesneye yönelik kavramları eksik olanlara Larman'ın kitabını okumaları önerildi.

## Kısa Tekrar Notları

- **Çözümleme (analiz):** Sistemi anlamak için parçalara ayırmak. Uygulama alanının dilini, kavramlarını, çalışanlarını ve işlemlerini öğrenmek.
- **Pazarlık (negotiation) aşaması:** Müşterinin farklı birimleri çelişen isteklerde bulunabilir. Ayrıca bazı istekler yapılamaz ya da çok pahalıdır. Bu aşamada istekler uzlaştırılır, bazılarından vazgeçilir.
- **Kullanım öyküsü belgesinin bölümleri:** birincil aktör, ilgililer ve beklentileri, ön koşul, son koşul, ana akış (main success scenario), alternatif akışlar.
- **Aktör:** Sistemle etkileşen varlık. İnsan, donanım ya da başka bir yazılım olabilir.
- **include:** A işi yapılırken B işi her zaman yapılır. Örnek: satış her zaman ödemeyi içerir.
- **extend:** A işi yapılırken bazı koşullarda B işine sapılır. Örnek: ödeme kredi kartıyla yapılırsa provizyon alınır; nakitte gerekmez.
- **Aday sınıf ölçütleri:** Kullanım senaryolarındaki isimler taranır. Aday sınıf ya saklanan bir bilgidir, ya gereken bir hizmettir, ya da problemin çözümü için gereken bir varlıktır.
- **CRC kartları (Class–Responsibility–Collaboration):** Her sınıfın sorumluluğunu ve işbirliği yaptığı sınıfları belirlemek için kullanılır.
- **Aggregation:** içi boş eşkenar dörtgen. **Composition:** içi dolu eşkenar dörtgen.
- **SRS belgesinde olması gerekenler:** UML kullanım şemaları, kullanım senaryoları, UML sınıf şemaları ve veritabanı için ER şemaları.
- **Etkinlik şeması sembolleri:** başlangıç (dolu daire), etkinlik sonu (halka içinde dolu daire), eylem (kenarları yuvarlatılmış dikdörtgen), karar (eşkenar dörtgen), fork/join (kalın senkronizasyon çubuğu).
- **Yeniden kullanılabilirlik:** Yazılımın bütününün ya da bir parçasının başka uygulamalarda da kullanılabilecek biçimde genel yazılması.

## Detaylı Açıklamalar

### Gereksinim analizi adımlarının nesneye yönelik çözümlemeye aktarılması

Hoca gereksinim analizinin adımlarını hatırlattı. Önce işin hedefi anlaşılır. Sonra uygulama alanı hakkında bilgi toplanır: mevcut uygulamalar, teknik literatür, müşteri anketleri, bugünkü ve gelecekteki gereksinimler. Bu bilgi, müşterinin dilini anlamak için gereklidir.

Modelleme (elaboration) adımında kullanım şemaları, kavramsal sınıf şemaları gibi modeller çizilir. Pazarlık (negotiation) adımında modeller gözden geçirilir ve taraflar uzlaşır. Uzlaşmadan sonra SRS belgesi ortaya çıkar. Tasarım bu belgeye dayanır.

### Kullanım öyküsü: market kasası örneği

Örnekte birincil aktör kasiyerdir. Satıcı ve müşteri ilgililerdir. Ana akış, satışın adım adım nasıl yürüdüğünü gösterir. Adımların sırası önemlidir.

Alternatif akış örnekleri:

- **Geçersiz barkod:** Sistem uyarı verir, kaydı reddeder ve yeni bir ürün okutulması istenir.
- **Müşteri bir ürünü bırakmak ister:** 7. adımda kasiyer toplamı bildirdikten sonra müşteri bir ürünü çıkarmak isteyebilir. Kasiyer ürünün barkodunu tekrar okutur ve güncel toplamı bildirir.

### Use case şeması sembolleri

- **Elips:** kullanım senaryosu (işlem).
- **Çöp adam:** insan aktör.
- **Dikdörtgen:** insan olmayan aktör, örneğin bir sensör ya da başka bir yazılım.

- **Düz çizgi:** aktör ile senaryo arasındaki etkileşim.
- **Kesikli ok, `<<include>>` / `<<extend>>`:** senaryolar arasındaki ilişki.

Örnek ilişkiler:

```
Kasiyer --- (Satış Yap)
(Satış Yap) --<<include>>--> (Ödeme Al)
(Provizyon Al) --<<extend>>--> (Ödeme Al)    [yalnız kartla ödemede]
```

Kahve sitesi örneğinde sipariş vermek, önceden bilgi girişi yapılmış olmasını içerir.

### Örnek alan modeli (domain model)

Slayttaki kavramsal sınıf şeması (ders 6, 00:27:00 karesi):

```
SatışKalemi (miktar: int)   1..* ---- 1   Satış (tarih: Date)
SatışKalemi  1 --"içerir"--> 1  Ürün (tanım: String, fiyat, kdv: double, barkod: String)
Ürün  * --"stoklanır"-- 1  Şube (ad, adres: String)
Şube  ◆---- Kasa                    (composition)
Kasa  --"kaydeder"--> Satış
Satış 1 --"içerir"--> 1  Ödeme (miktar: double)
```

Sınıfların üçüncü bölümü (işlemler) boştur. Çünkü bu bir kavramsal modeldir. Bir satış kalemi bir ürün tipini gösterir. Aynı defterden beş tane alınırsa tek bir satış kalemi olur ve miktarı 5'tir.

### Ardışıl şema (sequence diagram)

Nesneler şemanın üstünde, işleme giriş sırasıyla dizilir. Mesajlar ve metot çağrıları aşağıya doğru zaman sırasıyla gösterilir. Bir koşul altında başlayan işlemler ve döngüler de gösterilebilir. Kahve sitesi örnekleri bir önceki hafta işlenmişti.

### Etkinlik şeması (activity diagram)

Etkinlik şeması özellikle farklı ortamlarda ya da farklı aktörlerde yürüyen işleri göstermek için uygundur. Analizde kabaca çizilmeye başlanır. Tasarımda ardışıl şemayla birlikte ayrıntılandırılır.

**Uçak yolculuğu örneği:**

```
(başlangıç sinyali)
  -> Check-in işlemi
  -> [check-in başarısız?] -> (etkinlik sonu)
  -> [başarılı] -> FORK
        |-- Yolcu uçağa biner
        |-- Bagaj uçağa yüklenir
     JOIN
  -> Kapılar kapanır, uçak piste ilerler
```

Fork, bir akışın iki ortamda aynı anda yürüdüğünü gösterir. Join, iki kolun da bitmesini bekler ve akışı tek kola birleştirir.

**Kulvarlı (swimlane) örnek:** Yer hizmetleri çalışanları ve müşteri hizmetleri çalışanları ayrı sütunlarda gösterilir. Yer hizmetleri biniş listesini oluşturur. Müşteri hizmetleri yolcu bilgilerini veritabanına kaydeder. Her sütun, o aktörün yaptığı eylemleri ve aralarındaki ilişkiyi gösterir.

### Durum şeması (state diagram)

Durum şeması, bir nesnenin süreç boyunca hangi durumlardan geçtiğini ve geçişlerde hangi olayların olduğunu gösterir. Durumlar kenarları yuvarlatılmış dikdörtgenle, geçişler okla çizilir.

**Uçak yaşam döngüsü örneği:** sipariş verildi → teslim edildi → bakımda → kullanıma hazır → kullanımda → (arıza bulunursa) bakımda → … → kullanım dışı.

**Kasa satış örneği:**

```
[Boşta bekleme]
   --satış başlat--> [Satış kaydediliyor]  (barkodlar giriliyor)
        --ürün iptal--> [Satıştan ürün silme] --> [Satış kaydediliyor]
        --tüm ürünler girildi--> [Ödeme bekleniyor]
   --ödeme alındı--> [Fiş/fatura veriliyor]
   --> [Boşta bekleme]
   --kasayı kapat--> (son)
```

**Tasarım sınıf şeması**

Tasarım sınıf şeması, kavramsal şemaya işlemleri ekler. Örneğin ürünün var olup olmadığını döndüren mantıksal bir işlem, satış tarihini alma ve toplamı hesaplama işlemleri. Kasada barkod okuma özelliği olmalıdır. Kavramsal şemaya göre bazı küçük düzeltmeler de olabilir.

**Tasarım kalıpları**

Tasarım kalıbı, sık karşılaşılan bir probleme sistematik bir çözüm getiren prensiptir. Hoca bunu "aklın yolu birdir" diye özetledi. Her kalıbın, problemi akla getiren bir adı vardır. Kalıplar tasarım kalitesini, anlaşılırlığı ve yeniden kullanılabilirliği artırır, bakımı kolaylaştırır.

- **Yaratıcı (Creator):** Bir sınıftan nesne yaratma sorumluluğunu kime vereceğimizi söyler. Kahve sitesinde sipariş kalemi nesnesini sipariş yaratır.
- **Bilginin uzmanı (Information Expert):** Sorumluluk, o işi yapmak için gereken bilgiye sahip sınıfa verilir. Siparişin toplam bedelini sipariş sınıfı bilir. Her sipariş kalemi kendi bedelini bilir. Bunun için kahve nesnesiyle işbirliği yapar.
- **Az bağımlılık (Low Coupling):** Bir sınıftaki değişimin diğer sınıfları az etkilemesini sağlar. Çok bağımlılık bakımı ve testi zorlaştırır, yeniden kullanımı azaltır. Kahve sitesi örneğinde yaratıcı kalıbına göre ödemeyi müşteri yaratmalıdır. Ama sorumluluk sipariş sınıfına verilirse bağımlılık azalır. Kalıplar birlikte düşünülerek en iyi çözüm seçilir.
- **Denetçi (Controller):** Kullanıcı arayüzünün arkasında durur. Sistem olaylarını alır, kontrol eder ve işi yapacak nesneye yönlendirir. Tüm sistemi ya da bir kullanım senaryosunu temsil eden bir sınıf denetçi olabilir.
- **Yüksek uyum (High Cohesion):** Bir sınıfın sorumlulukları birbiriyle ilgili ve tek bir konuya yoğunlaşmış olmalıdır. Çok farklı işler yapan bir sınıfta uyum kötüdür.
- **Çok şekillilik (Polymorphism):** Tipe göre değişen davranışlar, çok şekilli metotlarla ilgili tiplere atanır.
- **Yapay sınıf (Pure Fabrication):** Uzman kalıbının çözümü uyum ve bağımlılık ilkeleriyle çelişirse, gerçek dünyada karşılığı olmayan bir sınıf yaratılabilir. Birbirine benzeyen işlevler bu sınıfta toplanır. Az kullanılması önerilir.
- **Dolaylılık (Indirection):** Değişebilecek iki birim arasına bir ara birim nesnesi konur. Örnek: farklı kredi kartı kuruluşlarının farklı onay mekanizmaları vardır. Uygulama tek bir çok şekilli arayüzle konuşur. Bu arayüzü her kuruluş için ayrı bir adaptör gerçekler.
- **Değişimlerden korunma (Protected Variations):** Değişmeye açık ve kararsız noktaların etrafındaki sorumluluklar arayüzlere atanır. Böylece bir birimdeki değişiklik diğerlerini en az etkiler.

**Kalıpların ardışıl şemada gösterilmesi: kahve sitesi**

- Yeni sipariş işleminde dış ortamdan gelen bilgiyi **denetçi** alır ve uygunluğunu kontrol eder.
- Sipariş nesnesini ve sipariş kalemlerini **yaratıcı** kalıbına göre sipariş yaratır.
- Seçilen kahvenin bilgisi kahve kataloğundan alınır. Burada **bilginin uzmanı** kalıbı devreye girer.
- Özel kahve siparişinde kalemler bir döngü içinde tek tek yaratılır.

Sınıf şemasında bir kahve sitesi çok sayıda sipariş alabilir, ama en az bir sipariş olmalıdır. Sipariş, fatura ve ödemeyle ilişkilidir. Sipariş kalemi fiyatı kahveden, miktarı kendisinden alır. Standart kalem ve özel kalem, sipariş kaleminden genelleştirme (generalization) ilişkisiyle türer.

### Etkileşim şemaları

Ardışıl şema ve işbirliği şeması birlikte etkileşim şemaları olarak adlandırılır. Ardışıl şema, nesnelerin olaya hangi sırayla girdiğini gösterir. Bu derste tasarım için sınıf şeması, ardışıl şema, etkinlik şeması ve durum şeması kullanılacak. İşbirliği şeması kullanılmayacak.

### Ders sonrası notlar

- **GoF ile GRASP karışıyor.** Hoca derste dokuz kalıbı (yaratıcı, uzman, az bağımlılık, denetçi, yüksek uyum, çok şekillilik, yapay sınıf, dolaylılık, değişimlerden korunma) "Dörtlü Çete" (Gang of Four, GoF) kalıpları olarak anlattı. Bu dokuz ilke aslında Craig Larman'ın **GRASP** (General Responsibility Assignment Software Patterns) ilkeleridir. GoF kalıpları ise Gamma, Helm, Johnson ve Vlissides'in 1994 tarihli *Design Patterns* kitabındaki 23 kalıptır: Singleton, Factory Method, Adapter, Observer, Strategy gibi. Hocanın dolaylılık örneğindeki kredi kartı adaptörü, GoF'un Adapter kalıbına karşılık gelir.
- Altyazıda yazar adları "Gama, Helm, Johnson ve Willisides" olarak geçiyor. Doğru yazım: Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides.
- **İşbirliği şemasında da sıra vardır.** Hoca işbirliği şemasında zaman sırasının önemli olmadığını söyledi. UML 2'de bu şemanın adı iletişim şeması (communication diagram) oldu. Bu şemada mesajlar numaralanır (1, 1.1, 2 ...) ve sıra bu numaralarla gösterilir. Fark, sıranın dikey eksenle değil, numarayla gösterilmesidir.
- **Etkinlik şemasında eylem kutusu.** Eylemler kenarları yuvarlatılmış dikdörtgenle çizilir. Düz dikdörtgen değildir.
- **Durum ile eylemin karıştırılmaması.** Durum şemasında kutular durumları (beklemede, ödeme bekleniyor) gösterir. Eylemler ise oklar üzerindeki olay ve eylem etiketleriyle gösterilir.
- **"Ekspert kalıbı".** Altyazıda "ekspert kalıbı" diye geçen kalıbın İngilizce adı Information Expert'tir.

**Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.